

Contents

Chapter 1. Deep Learning Introduction	2
1 Logistic Regression as a Neural Network	2
1.1 핵심 용어 정리 (Terminology)	2
1.2 Core Concepts: 신경망의 해부	2
1.2.1 1. 문제 정의: 이진 분류 (Binary Classification)	2
1.2.2 2. 뉴런의 구조: 선형 결합과 활성화	3
1.2.3 3. 비용 함수 (Cost Function): 왜 MSE가 아닐까?	3
1.3 Numerical Example: 손으로 풀어보는 로지스틱 회귀	4
1.4 Python Implementation (Vectorization)	4
1.5 자주 묻는 질문 (FAQ)	5

Chapter 1. Deep Learning Introduction (Completed)

(이전 단원에서 우리는 딥러닝이 무엇인지, 그리고 데이터가 어떻게 새로운 석유가 되었는지 배웠습니다. 이제 그 거대한 딥러닝이라는 기계를 구성하는 가장 작은 부품을 뜯어볼 차례입니다.)

1 Logistic Regression as a Neural Network

연결 고리

거대한 빌딩도 벽돌 한 장에서 시작하듯, 아무리 복잡한 AI(LLM, AlphaGo 등)도 결국 '뉴런 (Neuron)'이라는 작은 단위의 집합입니다. 이번 단원에서는 딥러닝의 가장 기초 단위인 로지스틱 회귀(Logistic Regression)를 하나의 신경망으로 해석하고, 그 내부 동작 원리를 완전히 해부합니다.

□ 단원 개요 (Overview)

이 단원은 딥러닝 학습의 '기초 체력'을 다지는 구간입니다.

- **목표:** 이진 분류 문제를 신경망 구조(Input → Linear → Activation)로 모델링합니다.
- **핵심:** 계산 그래프를 통해 순전파(Forward)와 역전파(Backpropagation)를 유도합니다.
- **이유:** MSE 대신 **Binary Cross-Entropy**를 비용 함수로 쓰는 이유를 이해합니다.
- **구현:** Python(NumPy)을 사용하여 for-loop 없는 **벡터화(Vectorization)** 코드를 작성합니다.

1.1 핵심 용어 정리 (Terminology)

용어	설명 (한 줄 정의)
특징 벡터 (Feature Vector, x)	예측을 위해 입력되는 데이터의 정보들 (예: 이미지의 픽셀값)
가중치 (Weight, w)	입력 정보가 결과에 미치는 중요도 (클수록 중요한 정보)
편향 (Bias, b)	입력과 상관없이 기본적으로 가지는 성향 혹은 임계값
시그모이드 (Sigmoid, σ)	계산된 점수를 0과 1 사이의 확률로 변환하는 활성화 함수
교차 엔트로피 (Cross-Entropy)	확률 분포 간의 차이를 측정하는 비용 함수 (틀릴수록 값이 커짐)

1.2 Core Concepts: 신경망의 해부

1.2.1 1. 문제 정의: 이진 분류 (Binary Classification)

한 줄 요약: 질문에 대해 YES(1) 또는 NO(0)로 답하는 문제입니다.

□ 고양이 탐지기 (직관적 비유)

당신이 사진을 보고 "이것은 고양이입니까?"라는 질문에 답해야 한다고 상상해봅시다.

- 입력(x): 사진 속의 털, 귀 모양, 눈동자 색깔 등의 단서들.
- 출력($\hat{y}$): "고양이일 확률은 80"

기술적 정의: n_x 차원의 입력 벡터 x 가 주어졌을 때, 출력 y 가 1일 확률 $\hat{y} = P(y = 1|x)$ 를 예측합니다.

1.2.2 2. 뉴런의 구조: 선형 결합과 활성화

로지스틱 회귀는 두 단계의 '생각 과정'을 거칩니다.

Step 1: 선형 결합 (Linear Function) - 점수 매기기

$$z = w^T x + b$$

- w (가중치): 각 단서가 얼마나 중요한지 결정합니다. (예: '뿔죽한 귀'는 고양이 판별에 중요함 $\rightarrow$ 높은 w)
- b (편향): 입력이 0이어도 기본적으로 갖는 점수입니다. (예: "나는 동물을 좋아해서 일단 고양이로 보고 싶어" $\rightarrow$ 높은 b)

Step 2: 활성화 함수 (Sigmoid) - 확률로 변환

$$a = \sigma(z) = \frac{1}{1 + e^{-z}}$$

선형 결합의 결과 z 는 $-\infty$ 에서 $+\infty$ 까지의 값을 가질 수 있습니다. 이를 0 ~ 1 사이의 확률로 압축하는 과정입니다.

- z 가 매우 크면 $\rightarrow a \approx 1$ (확신)
- z 가 0이면 $\rightarrow a = 0.5$ (반반)
- z 가 매우 작으면 $\rightarrow a \approx 0$ (아님)

1.2.3 3. 비용 함수 (Cost Function): 왜 MSE가 아닐까?

우리는 모델이 예측한 값(a)과 실제 정답(y)이 다르다면 모델을 '훈내줘야' 합니다. 그 벌점을 매기는 규칙이 비용 함수입니다.

MSE(평균 제곱 오차)를 쓰면 안 되나요? (오해 금지)

선형 회귀에서는 $MSE(\frac{1}{2}(\hat{y} - y)^2)$ 를 쓰지만, 로지스틱 회귀에서 이를 쓰면 비용 함수가 울퉁불퉁한(Non-Convex) 모양이 됩니다. 즉, 경사 하강법을 할 때, 진짜 최소점(Global Minimum)이 아닌 웅덩이(Local Optima)에 빠져 학습이 멈출 수 있습니다.

따라서 우리는 매끄러운 그릇 모양(Convex)을 보장하는 이진 교차 엔트로피(Log Loss)를 사용합니다.

$$L(a, y) = -(y \log(a) + (1 - y) \log(1 - a))$$

- 정답이 1($y = 1$)인데 예측을 0에 가깝게 하면($a \rightarrow 0$), $-\log(a)$ 때문에 비용이 무한대로 치솟습니다. (엄청난 벌점!)

1.3 Numerical Example: 손으로 풀어보는 로지스틱 회귀

수식을 이해하는 가장 좋은 방법은 숫자를 넣어보는 것입니다.

□ 시험 합격 예측 시나리오 (실전 계산)

상황: 학생의 '공부 시간(x_1)'과 '수면 시간(x_2)'으로 '합격($y = 1$)'을 예측합니다.

- 데이터: 공부 2시간($x_1 = 2$), 수면 5시간($x_2 = 5$). 실제 결과: 합격($y = 1$).
- 초기 파라미터: $w_1 = 0.1, w_2 = -0.1, b = 0.0$ (초기화 상태)

Step 1: 선형 계산 (Forward)

$$z = (w_1 \cdot x_1) + (w_2 \cdot x_2) + b$$

$$z = (0.1 \cdot 2) + (-0.1 \cdot 5) + 0 = 0.2 - 0.5 = -0.3$$

Step 2: 활성화 (Sigmoid)

$$a = \frac{1}{1 + e^{-(-0.3)}} = \frac{1}{1 + 1.349} \approx 0.425$$

→ 모델은 합격 확률을 **42.5%**로 예측했습니다. (실제는 합격인데, 예측이 틀렸네요!)

Step 3: 비용 계산 (Loss)

$$L = -(1 \cdot \log(0.425) + 0 \cdot \log(\dots)) \approx -(-0.855) = 0.855$$

→ 벌점은 **0.855**입니다.

Step 4: 역전파 (Backward) - 학습의 핵심 우리는 정답($y = 1$)에 가까워지도록 w 를 수정해야 합니다. 여기서 마법의 공식 $dz = a - y$ 가 등장합니다.

$$dz = a - y = 0.425 - 1 = -0.575$$

이제 가중치에 대한 기울기(dw)를 구합니다.

$$dw_1 = x_1 \cdot dz = 2 \cdot (-0.575) = -1.15$$

$$dw_2 = x_2 \cdot dz = 5 \cdot (-0.575) = -2.875$$

$$db = dz = -0.575$$

Step 5: 파라미터 업데이트 (Gradient Descent) 학습률 $\alpha = 0.1$ 이라고 가정합니다.

$$w_1 \leftarrow w_1 - \alpha \cdot dw_1 = 0.1 - 0.1(-1.15) = 0.1 + 0.115 = 0.215$$

$$w_2 \leftarrow w_2 - \alpha \cdot dw_2 = -0.1 - 0.1(-2.875) = -0.1 + 0.2875 = 0.1875$$

결과 해석: w_1 (공부 시간 중요도)이 0.1에서 0.215로 증가했습니다. 즉, 모델은 "공부를 많이 할수록 합격한다"는 것을 배웠습니다!

1.4 Python Implementation (Vectorization)

이론을 실제 코드로 옮겨봅시다. 여기서 중요한 것은 'for' 루프를 쓰지 않는 **벡터화**입니다.

```

1 import numpy as np
2
3 def propagate(w, b, X, Y):
4     """
5     Arguments:
6     w -- 가중치 (n_x, 1)
7     b -- 편향 (scalar)
8     X -- 입력 데이터 (n_x, m) -> 은 m 데이터 개수
9     Y -- 실제 레이블 (1, m)
10    """
11    m = X.shape[1]
12
13    # --- Forward Propagation 순전파() ---
14    # 행렬 연산으로 개의 m 데이터를 한 번에 계산 (Vectorization)
15    Z = np.dot(w.T, X) + b
16    A = 1 / (1 + np.exp(-Z)) # Sigmoid
17
18    # Cost 계산
19    cost = -1/m * np.sum(Y * np.log(A) + (1-Y) * np.log(1-A))
20
21    # --- Backward Propagation 역전파() ---
22    # 수학적으로 유도된 공식: dZ = A - Y
23    dZ = A - Y
24
25    # Gradients 계산
26    dw = 1/m * np.dot(X, dZ.T) # 차원 확인: (n, m) * (m, 1) = (n, 1)
27    db = 1/m * np.sum(dZ)
28
29    return dw, db, cost

```

Listing 1: Vectorized Logistic Regression

주의: Rank-1 Array Pitfall (오해 금지)

NumPy에서 'a = np.random.randn(5)'는 '(5,)' 형태를 가집니다. 이는 행 벡터도 열 벡터도 아니어서 전치('T')를 해도 모양이 바뀌지 않아 버그를 유발합니다. 반드시 'a = np.random.randn(5, 1)' 처럼 차원을 명시하십시오.

1.5 자주 묻는 질문 (FAQ)

Q1. 편향(Bias) b 는 왜 필요한가요? 없으면 안 되나요?

A. 원점을 반드시 지나야 한다는 제약이 생깁니다. 예를 들어, 공부를 하나도 안 했어도 ($x = 0$) 합격할 확률이 0이 아닐 수 있습니다. b 는 그래프를 좌우로 이동시켜 데이터에 더 잘 맞도록 해주는 '유연성'을 제공합니다.

Q2. 초기화할 때 w 를 0으로 뒀도 되나요?

A. 로지스틱 회귀에서는 가능합니다. 비용 함수가 볼록(Convex)하기 때문에 어디서 시작하든 바닥(최적해)으로 굴러갑니다. 하지만 나중에 배울 심층 신경망(Deep Network)에서는 절대 0으로 초기화하면 안 됩니다(대칭성 문제).

Q3. 학습률(Learning Rate)이 너무 크면 어떻게 되나요?

A. 보폭이 너무 커서 최적점을 지나쳐 버리거나(Overshooting), 영원히 수렴하지 않고 발산할 수 있습니다. 반대로 너무 작으면 학습 속도가 너무 느려집니다.

□ 단원 요약 (Chapter Summary)

1. 로지스틱 회귀는 딥러닝의 가장 작은 단위인 **1-Layer Neural Network**이다.
2. 구조: $z = w^T x + b$ (선형) $\rightarrow a = \sigma(z)$ (비선형 활성화).
3. 학습: 이진 교차 엔트로피를 최소화하는 방향으로 경사 하강법을 수행한다.
4. 구현: m 개의 데이터를 'for'문 없이 처리하기 위해 **Vectorization(행렬 연산)**을 사용한다.

다음 단원 예고

축하합니다! 여러분은 이제 신경망의 '뇌세포' 하나를 완벽하게 만들 수 있습니다. 다음 장 [**Chapter 3. Shallow Neural Networks**]에서는 이 세포들을 옆으로 나란히 연결하고, 뒤로 층층이 쌓아서 더 복잡한 문제를 해결하는 은닉층(**Hidden Layer**)의 마법을 배워보겠습니다.